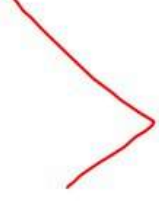


Final Review

About the final exam



- Grading policy:

- Total = Participation $\times$ 40% + Final exam $\times$ 60%
- Participation = Lectures + Labs

- Final exam:

- Semi-open-book exam

1. This examination is conducted as a semi-open-book exam, allowing each student to bring one handwritten A4-sized note sheet, which may contain writing on both sides.
2. Calculators are not allowed.
3. A blank scratch paper will be provided to each student.

Data representation and operation

System	Base	Symbols	Examples
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	2345.56
Binary	2	0, 1	$(1001.11)_2$
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	$(156.23)_8$
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F	$(A2C.A1)_{16}$

Handwritten notes showing binary representations and powers of 2:

$2^7 \leftarrow 1$
 $2^6 \leftarrow 1$
 $2^5 \leftarrow 1$
 $2^4 \leftarrow 1$
 $2^3 \leftarrow 0$
 $2^2 \leftarrow 1$
 $2^1 \leftarrow 1$
 $2^0 \leftarrow 1$
 0.1B

position	7	6	5	4	3	2	1	0	-1	-2	-3
	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	$2^{(-1)}$	$2^{(-2)}$	$2^{(-3)}$
value	128	64	32	16	8	4	2	1	0.5	0.25	0.125
		1					1				

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

2. Conversion

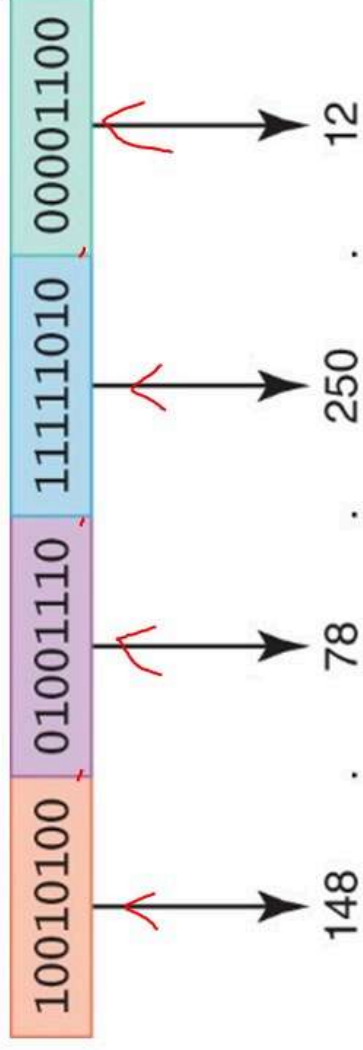
- Converting the integral part
 - examples (binary): division method

98=(1100010)B

	98	remainder	
2	49	0	Low position
2	24	1	
2	12	0	
2	6	0	
2	3	0	
2	1	1	
	0	1	High position

$$\begin{aligned}
 98 &= 2 \times 49 + 0 \\
 &= 2 \times (2 \times 24 + 1) + 0 \\
 &= 2 \times (2 \times (2 \times 12 + 0) + 1) + 0 \\
 &= 2 \times (2 \times (2 \times (2 \times 6 + 0) + 0) + 1) + 0 \\
 &= 2 \times (2 \times (2 \times (2 \times (2 \times 3 + 0) + 0) + 1) + 0) + 0 \\
 &= 2 \times (2 \times (2 \times (2 \times (2 \times (2 \times 1 + 1) + 0) + 0) + 1) + 0) + 0 \\
 &= 2^6 + 2^5 + 2^1
 \end{aligned}$$

IP address

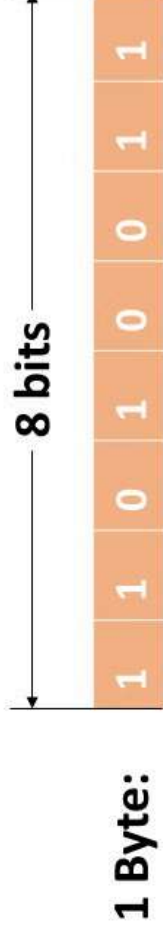


3. Data Operations

- 2. Arithmetic Operations (Binary)
 - Addition: ✓
 - Addition is the *foundation* of arithmetic operations.
 - Subtraction: ✓
 - Subtraction can be transformed into addition.
 - Multiplication: ✓
 - Multiplication is a repeated addition operation.

• $11+1011=$

• $11*1011=$



- Bit – Binary Digit (smallest unit of data in a computer)
 - 1 Byte = 8 Bits
 - Enough space for: a single letter or symbol.
 - 1 Kilobyte(KB) = 1024 Bytes = 2^{10} Bytes $\approx$ 1000 Bytes
 - Enough space for: about 1/3 page of text.
 - 1 Megabyte(MB) = 1024 Kilobytes = 2^{10} Kilobytes $\approx$ 1000 Kilobytes
 - Enough space for: about one book, or one photo, or one minute of music.
 - 1 Gigabyte(GB) = 1024 Megabytes = 2^{10} Megabytes $\approx$ 1000 Megabytes
 - Enough space for: about 1000 books, or 1000 photos, or 16 hours of music.
 - 1 Terabyte(TB) = 1024 Gigabytes = 2^{10} Gigabytes $\approx$ 1000 Gigabytes
- PB/EB/ZB/YB...

ASCII	@	<u>A</u>	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
-------	---	----------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

ASCII

- E.g. Encode the sentence "This C program has bugs." (without quotes) using the ASCII table (attached at the end of the examination paper). Write down the code for each character in the table as below (in the order from left to right, top to bottom).

ASCII

- E.g. Decode the text represented by the ASCII code given as below, and write it down in the subsequent table:

```
01001001 00100111 01101101 00100000 01110111 01110010 01101001 01110100 01101001  
01101110 01100111 00100000 01110100 01101111 00100000 01111001 01101111 01110101  
00100000 01100110 01101111 01110010 00100000 01101000 01100101 01101100 01110000  
00101110
```


Pseudocode to C and Python

```
If x<0  
    y ← x  
    x ← x+1
```

PYTHON:

```
if x < 0:  
    y = x  
    x = x + 1
```

C:

```
if (x < 0) {  
    y = x;  
    x = x + 1;  
}
```

```
While i ≤ 100
```

```
    s ← s − i  
    i ← i + 1
```

PYTHON:

```
While i<=100:  
    s=s-i  
    i=i+1
```

C:

```
While(i<=100)  
{  
    s=s-i;  
    i=i+1;  
}
```

E.g.

You are required to both draw a **flow chart** and write down its equivalent **pseudocode** for each of the following three tasks:

- (1) Input an integer number n (ranging from 0 to 100) from the keyboard, test if it is larger than or equal to 80. If yes, then output "A"; otherwise, Output "B".
- (2) First input an integer n , and then n integer numbers (ranging from 0 to 100) to an array `grade[]` from the keyboard. Output the **average** of all these n numbers in the array.
- (3) Output all indices corresponding to the minimal value of the numbers stored in the array created in (2).
For example, if the array is [77, 56, 81, 93, 61, 77, 56, 90], then output indices 2, 7 (corresponding to the minimum 56).

- Hardware ✓
- Operating system ✓
- Networks ✓

